

Apertus Sovereign Operations Bridge

Technical report - 8 October 2026

Track: 2B - Apertus Adoption / Own Project. **Event:** Hack Apertus Online. **Team:** SPINNENNETZ-DNA. **Participant:** Stefan Ambrosini. **Application baseline:** commit 695a48d62973b11468a3fe1591da223668c6f30a.

1. Summary

Business users need to distinguish a plausible model answer from a result supported by complete, consistent sources and appropriate authorization. This standalone prototype uses Apertus for semantic extraction and draft preparation, followed by deterministic checks for conflicts, missing facts, embedded instructions and protected actions. The model cannot authorize its own external action. Five synthetic live scenarios using CSCS-hosted Apertus v1.5 70B produced their expected control outcomes on 3 October 2026; the complete sanitized evidence is public [1]. These are five demonstrations, not a population-level accuracy estimate.

2. Architecture

Request and synthetic source documents -> Apertus structured output -> deterministic controls -> explicit outcome and evidence -> browser/CLI readback.

Python's HTTP service renders the browser UI and accepts a scenario request. The OpenAI-compatible client calls Apertus once with a JSON-output prompt. The schema carries facts, source IDs, missing information, untrusted instructions, a proposed response and requested actions. The independent control layer emits VERIFIED, HUMAN_REVIEW or BLOCKED. A caller-observed protected action is retained even if the model omits it. SHA-256 binds the input documents and request to the evidence. The prototype prepares results; it has no email-send, printer, Office, payment or contract execution tool.

Target architecture: on-premise

The application can run in a Docker container under the organization's administration, configured to an operator-managed Apertus endpoint through LLM_NAME, LLM_BASE_URL and LLM_API_KEY. The endpoint must provide OpenAI-compatible chat completions. Hosting model weights and GPU inference is a prerequisite, not bundled in this CPU application container. A fully self-hosted deployment, hardware sizing and an air-gapped model stack have not been tested.

The demonstrated deployment uses a public Render application and authorized Swiss CSCS inference. It does not establish Swiss residency for the whole application stack. No claim of a verified all-Swiss deployment is made. At build time Docker downloads the Python base image. At runtime live mode contacts the configured inference endpoint; the app otherwise uses Python's standard library. Mock mode is an explicitly labelled, offline control demonstration.

3. Use of Apertus

Verified model: swiss-ai/Apertus-v1.5-70B. **Serving:** authorized CSCS hosted inference, <https://api.inference.cscs.ch/v1>. **Role:** semantic extraction and response preparation, with no fine-tuning, secondary judge model or model-issued authorization.

The client uses /chat/completions, temperature 0, max_tokens 1200 and JSON-object response format. It parses a complete JSON object, including replies surrounded by code fences or prose. Prompts are in src/app/prompt.py; the schema, fixtures and deterministic checks are in src/app/schema.py, fixtures.py and control.py. Deterministic checks cannot compensate for every possible extraction error. Development used ChatGPT/Codex assistance; neither is a runtime dependency of this public prototype.

4. Data

The project uses original, synthetic documents about an invented office product, delivery policy and customer request. There are five fixture variants. No customer, legal, family, email or other private SPINNENNETZ-DNA records are included. The data/ directory is below 100 MB. Fixtures are embedded in the existing Apache-2.0 source; live model responses are in the public evidence file [1]. No external dataset is submitted.

5. Evaluation

The task is to produce the correct control state for each defined scenario. Expected states are specified in advance. The mock baseline validates the deterministic checks; the live run validates one end-to-end model pass over the same synthetic cases. Both achieved 5/5 expected outcomes. Every recorded live request returned HTTP 200. No protected action was executed.

Scenario	Expected and live outcome	Live request latency
Complete sources	VERIFIED	3442.22 ms
Missing information	HUMAN_REVIEW	3642.48 ms
Conflicting sources	HUMAN_REVIEW	4434.80 ms
Embedded instruction	BLOCKED	4371.27 ms
Protected action	HUMAN_REVIEW	3937.93 ms

These latencies come from the single published run on 3 October, not a new benchmark. Median: 3937.93 ms, range: 3442.22-4434.80 ms. No throughput, cost, 8B comparison, repeated-seed robustness or representative SME dataset evaluation has been completed. Temperature 0 does not guarantee deterministic hosted model output.

6. Limitations and scope of the supplement

The public live evidence covers five synthetic cases. It does not establish general correctness, universal prompt-injection resistance, unrestricted autonomy or production readiness. VERIFIED means that the implemented deterministic checks found no blocker; it is not a factual guarantee. Endpoint access, availability and model behavior can change. Hosted inference credentials are provided by the operator and never committed or included in evidence.

On 8 October, a separate private phone/HP integration environment verified phone-to-PC communication after a phone-session restart, two owner-confirmed physical printouts, and five native Office jobs: Word creation and append-to-copy, Excel creation and cell edits in a new copy, and PowerPoint creation. The Office files were saved, reopened in their native applications and checked; source copies were preserved. These jobs used bounded direct worker actions without a coding-agent invocation. These are maintainer-verified integration results. Their private code, device details and documents are not part of this submission. They do not demonstrate Apertus-controlled Office or printing.

General mission recovery, automatic cross-system handoff, closed-lid operation and remote wake remain unverified in that integration environment. Its cloud services and Windows/Office dependencies do not form part of the public prototype's sovereignty claim.

7. Reproducibility

Clone the public repository [2]. Docker and Make are required for the judge package. The app runs on a CPU; live inference runs on the selected endpoint's hardware, which was not measured. The application source snapshot matches the public baseline above; tests detect drift from the root runtime. The dated live results are preserved at immutable evidence commit 587522e1da34e5de17d40cde80dcb30cdf4ec72e [1].

From the repository root or track_2b/: set LLM_NAME=swiss-ai/Apertus-v1.5-70B, LLM_BASE_URL=<https://api.inference.cscs.ch/v1> and the authorized LLM_API_KEY in the environment, then run make run. Open <http://127.0.0.1:8080>. Live mode fails when its model/endpoint configuration is missing and never silently switches to mock. An on-premise endpoint can be supplied instead. Credentials are not passed as literal command-line values.

Run make demo for the explicit mock/control demonstration, or python -m unittest discover -s tests -v at the repository root for the existing tests. On 8 October, 30 existing tests and three package tests passed. The judge-package workflow successfully built the Docker image, started it through make run and verified all five mock HTTP outcomes [6]. It used no inference secrets; this is not another live Apertus run.

The original hosted app remains at [3]. The root Makefile forwards to track_2b/. The compatibility snapshot is refreshed with scripts/sync_judge_package.py and checked by tests. The package is an adaptation of the existing public repository to the current track layout, not a new project copied from private infrastructure.

8. Next steps and submission status

The next technical steps are repeated source-level evaluation, additional synthetic SME workflows, an 8B/70B resource comparison, and a fully self-hosted target deployment. These remain prospective. The existing Devpost description has been supplemented with the scoped 8 October integration results.

The organizers require the separate official submission portal [4]. A Devpost submitted badge does not prove receipt there. Portal receipt and the final two-minute prototype demo requirement must be checked before the deadline of 16 October 2026, 12:00 CEST. This report does not claim that an official portal submission has been accepted.

Media

The existing Devpost main video is <https://youtu.be/Xzft-gigylA>. Its player shows 1 minute 8 seconds, within the length limit. Its title and available transcript describe a general SPINNENNETZ-DNA promotional film; a concrete demonstration of the Apertus prototype has not been established. That content gap remains open. The additional architecture animation [5] is 2 minutes 26 seconds and does not replace the required maximum two-minute prototype demo.

Track 2B contribution

Purposeful AI use: Apertus performs language analysis, with authorization outside the model. Technical rigour: structured schema, source IDs, caller-side action retention, automated checks and dated evidence. Value: one model call per scenario and inspectable, reusable control logic; operating cost savings are not measured. Sovereign deployability: configurable open-model endpoint and an on-premise application path, with deployment limits stated explicitly. Feasibility: public source, browser UI, CLI, Docker package and live inference evidence.

References

[1] Live evidence: <https://github.com/ambrosinistefan8-stack/apertus-sovereign-operations-bridge/blob/587522e1da34e5de17d40cde80dcb30cdf4ec72e/evidence/live-cscs-20261003.json>

[2] Source and tests: <https://github.com/ambrosinistefan8-stack/apertus-sovereign-operations-bridge>

[3] Public demo: <https://apertus-sovereign-operations-bridge.onrender.com>

[4] Official submission portal: <https://hackapertus.ch/online-hack/submissions>

[5] Additional architecture explainer: <https://scrimba.com/explain/guide0fr35d7gp?fullscreen=1>

[6] Docker/Make verification: <https://github.com/ambrosinistefan8-stack/apertus-sovereign-operations-bridge/actions/runs/37840042147>

Requirements reviewed: https://github.com/HackApertus/project-template/tree/main/track_2b

License

This report: Creative Commons Attribution 4.0 (CC-BY-4.0). Source code remains Apache-2.0. The private integration environment is excluded from the submitted code and data; the public supplement describes only its limited engineering findings.